MarketSync

Real-Time Multi-Market Paper Trading Platform

DOCUMENT TYPE Technical User Manual	VERSION 1.0.0
DATE April 2026	FRAMEWORK Next.js 16 (App Router)
DATABASE Prisma Postgres + TimescaleDB	MARKETS India · US · Hong Kong

INTERNAL TECHNICAL DOCUMENTATION

Table of Contents

1. Introduction & Overview 3

1.1 What is MarketSync? 3

1.2 Key Features 3

1.3 Technology Stack 3

2. System Architecture 4

2.1 High-Level Architecture 4

2.2 Database Architecture (Single TimescaleDB) 4

2.3 Request Flow Diagram 5

2.4 Project File Structure 5

3. Setup & Configuration 6

3.1 Prerequisites 6

3.2 Environment Variables 6

3.3 Database Setup 6

3.4 Running Locally 7

4. Authentication 7

4.1 Sign Up 7

4.2 Login 7

4.3 JWT & Session Management 7

4.4 Route Protection 8

5. Market Watchlists 8

5.1 India (NSE) Watchlist 8

5.2 US Market Watchlist 9

5.3 Hong Kong Watchlist 9

5.4 Candlestick Charts 9

6. Paper Trading Engine 10

6.1 Placing Orders 10

6.2 Order Types & Statuses 10

6.3 Order Execution Logic 11

7. Portfolio Management 11

7.1 Positions & Holdings 11

7.2 P&L Calculation 12

8. Price Alerts 12

9. User Profile 13

10. API Reference 13

11. Data Providers & Market Data 15

12. Deployment (Vercel) 16

13. Troubleshooting 17

1 Introduction & Overview

What MarketSync is and what it can do

1.1 What is MarketSync?

MarketSync is a full-stack, real-time multi-market trading dashboard built on Next.js 16. It lets registered users monitor live price data from three major global exchanges — India (NSE), United States (NYSE / NASDAQ), and Hong Kong (HKEX) — and execute paper trades (simulated orders) with a virtual starting balance of **₹1,00,000**.

All price data is streamed over WebSocket connections directly in the browser. Candlestick (OHLCV) data and all transactional user data (accounts, orders, positions, alerts) are persisted in a single **TimescaleDB** database on TigerData Cloud — accessed via Prisma ORM for app tables and raw SQL for the candle hypertable.

> ✅ **Default Market: India (NSE)**
>
> After login, users are redirected to the India watchlist. The India market and ₹ (Rupee) currency are shown by default throughout the application.

1.2 Key Features

📊 Live Market Data

- Real-time NSE price feeds via Upstox WebSocket
- US & HK prices via AllTick WebSocket
- Bid / Ask / Spread / Mid price display
- Market status clock (timezone-aware)
- Connection health badges

📈 Candlestick Charts

- Interactive OHLCV charts (lightweight-charts)
- Multiple timeframes: 1m, 5m, 15m, 30m, 1h, 1d
- Period filters: 1W, 1M, 3M, 6M, 1Y
- TimescaleDB caching (stale-while-fresh)
- Auto-fallback to daily interval when market closed

🛒 Paper Trading

- Market & Limit order types
- Buy / Sell with balance validation
- Weighted-average cost tracking
- Order history with status tracking

💼 Portfolio & P&L

- Live unrealized P&L per position
- Realized P&L ledger on each sale
- Total portfolio value (cash + holdings)
- Market allocation breakdown

- Multi-market order management
- Filter by market (All / India / US / HK)

🔔 Price Alerts

- Conditions: ABOVE / BELOW / EQUAL
- Actions: NOTIFY / BUY / SELL
- Auto-execute orders when triggered
- Full alert history & status tracking

👤 User Profile

- Initials avatar with account stats
- Edit display name
- Change password (bcrypt-secured)
- Account metadata (ID, email, join date)

1.3 Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Framework	Next.js (App Router + Turbopack)	16.2.3	Full-stack React SSR/SSG
UI Library	React	19.2.4	Component rendering
Styling	Tailwind CSS	4.x	Utility-first CSS
Charts	lightweight-charts	5.1.0	Candlestick & line charts
ORM	Prisma + PrismaPg adapter	7.7.0	Relational DB queries (app tables)
Database	TimescaleDB (TigerData cloud)	PostgreSQL 16 + extension	All data: Users, Orders, Positions, Candles
Cron / Prefetch	GitHub Actions (scheduled workflow)	—	Pre-fetch daily candles every 5 min
Auth	Jose (JWT) + bcryptjs	—	Token-based sessions
Market Data (India)	Upstox API (WebSocket + REST)	v2	NSE live & historical data
Market Data (US/HK)	AllTick API (WebSocket + REST)	—	NYSE/NASDAQ/HKEX data
Deployment	Vercel	—	Hosting + serverless functions

2 System Architecture

How the components connect and communicate

2.1 High-Level Architecture

Browser (Client)

- React components (Next.js App Router)
- WebSocket streams (AllTick / Upstox)
- lightweight-charts canvas
- JWT cookie (httpOnly, 7-day)

Next.js Server

- API Route Handlers (app/api/)
- proxy.ts — auth middleware
- Prisma client (relational data)
- pg Pool (TimescaleDB)
- Paper trading engine (lib/trading.ts)

Database (Single)

- **TimescaleDB** (TigerData cloud)
- → Users, Orders, Positions, Alerts, PnL
- → Candle OHLCV hypertable
- App tables via Prisma ORM
- Candles via raw SQL (`pg` Pool)

2.2 Database Architecture (Single-DB)

MarketSync uses a **single TimescaleDB instance** on TigerData Cloud for all data:

TABLE GROUP	TABLES	ACCESS METHOD
App tables	User, Order, Position, PnlEntry, PriceAlert, Setting	Prisma ORM (`@prisma/client` via `@prisma/adapter-pg`)
Candle hypertable	Candle (market, symbol, interval, time, OHLCV, fetchedAt)	Raw SQL via `pg` Pool (`lib/timescale.ts`)

Why TimescaleDB for Everything?

TimescaleDB extends PostgreSQL with automatic hypertable partitioning (30-day chunks), columnar compression, and fast time-range queries — ideal for OHLCV candle storage. Prisma's `@prisma/adapter-pg` allows the ORM to connect directly to the same instance, eliminating the need for a separate Prisma Postgres database.

2.3 Request Flow Diagram

```
USER → Browser loads page (Next.js App Router) │ ▼ proxy.ts [Auth Gate — runs on every request] ├─ Public route? (/login, /signup, /api/auth/*) → PASS THROUGH ├─ No ms_token
```

```
cookie? → REDIRECT to /login ├─ Invalid JWT? → CLEAR cookie + REDIRECT to /login └─ Valid
JWT → PASS THROUGH to destination │ ▼ Next.js Page Component (Server or Client Component) │
├──── [Market data] WebSocket (AllTick / Upstox) │ Real-time tick prices → React state → UI
update │ └──── [API Route Handler] (app/api/**/) │ ├─ Read JWT from cookie → verify →
extract userId │ ├─ Trading operations → TimescaleDB (Prisma ORM) │ User | Order | Position
| PnlEntry | PriceAlert | └─ Candle data → TimescaleDB (pg Pool) getNewestFetchedAt()
(cache freshness check) readCandles() (read from hypertable) writeCandles() (bulk
INSERT/UPDATE via unnest()) │ ├─ Cache HIT + fresh → return DB candles └─ Cache MISS /
stale → fetch Upstox / AllTick → writeCandles() to cache → return live candles GitHub
Actions cron → GET /api/cron/prefetch (every 5 min) Prefetches daily candles for all
instruments Upstox (India · 1d) + AllTick (US · 1d, HK · 1d) → writeCandles() ON CONFLICT
DO UPDATE
```

2.4 Project File Structure

```
MarketSync/
├── app/
│   ├── api/
│   │   ├── auth/
│   │   │   ├── login/route.ts       # POST — issue JWT
│   │   │   ├── logout/route.ts      # POST — clear cookie
│   │   │   ├── me/route.ts          # GET  — current user
│   │   │   └── signup/route.ts      # POST — register user
│   │   ├── alerts/route.ts          # GET/POST alerts
│   │   ├── alerts/[id]/route.ts     # PATCH/DELETE alert
│   │   ├── alerts/check/route.ts    # POST — trigger check
│   │   ├── orders/route.ts          # GET/POST orders
│   │   ├── orders/[id]/route.ts     # GET/PATCH order
│   │   ├── portfolio/route.ts       # GET portfolio summary
│   │   ├── profile/route.ts         # GET/PATCH user profile
│   │   ├── india/kline/route.ts     # GET India candles (Upstox)
│   │   ├── us/kline/route.ts        # GET US/HK candles (AllTick)
│   │   ├── cron/prefetch/route.ts   # GET — prefetch daily candles (cron)
│   │   └── public/india-preview/   # GET — login page live NSE preview (no auth)
│   ├── components/
│   │   ├── NavBar.tsx               # Top navigation bar
│   │   ├── WatchList.tsx            # US/HK live price table
│   │   ├── IndiaWatchList.tsx       # NSE live price table
│   │   ├── CandleChartModalV2.tsx   # Fullscreen chart modal
│   │   ├── MiniChart.tsx            # Sparkline in watchlist rows
│   │   ├── MarketClock.tsx          # Timezone-aware market clock
│   │   └── ConnectionBadge.tsx      # WS connection indicator
│   ├── india/ us/ hk/               # Market watchlist pages
│   ├── orders/                      # Order management page
│   ├── portfolio/                   # Portfolio dashboard
│   ├── alerts/                      # Price alert management
│   ├── profile/                     # User profile page
│   ├── login/ signup/               # Auth pages
│   └── layout.tsx                   # Root layout
├── lib/
```

```
│   ├── prisma.ts                  # Singleton Prisma client
│   ├── timescale.ts               # TimescaleDB helpers
│   ├── auth.ts                    # JWT sign/verify
│   ├── session.ts                 # Session extraction
│   ├── trading.ts                 # Paper trading engine
│   ├── tick-data/                 # AllTick WS manager
│   └── upstox-tick-data/          # Upstox WS manager
├── config/
│   └── markets.ts                 # Market definitions
├── prisma/
│   ├── schema.prisma              # Prisma schema
│   └── migrations/                # DB migrations
├── scripts/
│   ├── init-timescale.sql         # TimescaleDB hypertable setup
│   └── setup-timescaledb.sql      # App table DDL (run once)
├── .github/
│   └── workflows/
│       └── prefetch-candles.yml   # GitHub Actions cron (every 5 min)
├── proxy.ts                       # Auth middleware
└── .env.local                     # Environment variables (gitignored)
```

3 Setup & Configuration

Prerequisites, environment variables, and running the project

3.1 Prerequisites

REQUIREMENT	VERSION	NOTES
Node.js	20+	LTS recommended
npm / pnpm / bun	Any	npm used in scripts
Prisma Postgres account	—	Free tier at prisma.io
TimescaleDB account	—	TigerData cloud (tigerdata.com)
Upstox API token	—	OAuth token for India data
AllTick API token	—	For US and HK market data

3.2 Environment Variables

Create a `.env.local` file in the project root (gitignored) with the following variables:

VARIABLE	REQUIRED	DESCRIPTION
`DATABASE_URL`	REQUIRED	TimescaleDB connection string — used for both Prisma ORM and candle pg Pool
`JWT_SECRET`	REQUIRED	Secret key for JWT signing (min 32 chars)
`CRON_SECRET`	REQUIRED	Bearer token for GitHub Actions → `/api/cron/prefetch` auth
`UPSTOX_ACCESS_TOKEN`	FOR INDIA	Bearer token from Upstox OAuth flow (expires daily)
`NODE_TLS_REJECT_UNAUTHORIZED`	FOR INDIA	Set to `0` to bypass Upstox TLS certificate issue
`NEXT_PUBLIC_ALLTICK_TOKEN`	FOR US/HK	AllTick API token (exposed to client for WebSocket)

```
# .env.local example
DATABASE_URL="postgres://user:pass@your-timescale-host:5432/dbname?sslmode=require"
JWT_SECRET="your-long-random-secret-key-here-2026"
CRON_SECRET="your-cron-bearer-secret"
UPSTOX_ACCESS_TOKEN="your-upstox-bearer-token"
NODE_TLS_REJECT_UNAUTHORIZED="0"
```

```
NEXT_PUBLIC_ALLTICK_TOKEN="your-alltick-token"
```

3.3 Database Setup

A single TimescaleDB instance on TigerData Cloud holds all data. Setup steps:

1. Create a TimescaleDB service at **tigerdata.com** (or Timescale Cloud).
2. Copy the PostgreSQL connection string to `DATABASE_URL` in `.env.local`.
3. Create the Candle hypertable: `psql "$DATABASE_URL" -f scripts/init-timescale.sql`
4. Create app tables (User, Order, Position, etc.) by running the DDL: `psql "$DATABASE_URL" -f scripts/setup-timescaledb.sql`
5. Generate the Prisma client: `npx prisma generate`

> ℹ️ **Do Not Use prisma migrate deploy**
>
> Because the Candle hypertable is created outside Prisma, running `prisma db push` or `prisma migrate deploy` on a non-empty TimescaleDB would attempt to drop and recreate tables. Use the provided SQL scripts instead.

3.4 Running Locally

```
# Install dependencies
npm install

# Generate Prisma client (must run before build)
npx prisma generate

# Start development server (Turbopack)
npm run dev

# Production build
npm run build   # = prisma generate && next build
npm start
```

The application will be available at **http://localhost:3000**. The root route (`/`) automatically redirects to `/india`.

4 Authentication

Sign up, login, JWT sessions, and route protection

4.1 Sign Up

New users register at **/signup**. The registration form requires:

- **Full Name** — displayed in the profile avatar and navigation
- **Email Address** — must be unique; used for login
- **Password** — hashed with `bcryptjs` (10 rounds) before storage

On successful registration, the user receives a **₹1,00,000 starting balance** and is redirected to the India watchlist.

> ℹ️ **Password Security**
> Passwords are never stored in plain text. bcrypt hashing with a cost factor of 10 is applied before the record is written to the database. Password comparison on login uses `bcrypt.compare()`.

4.2 Login

Existing users log in at **/login** with their email and password. On success:

1. Server verifies the password hash with bcrypt.
2. A **JWT token** is signed using `JWT_SECRET` with a 7-day expiry.
3. The token is set as an **httpOnly cookie** named `ms_token`.
4. The browser is redirected to `/india`.

To log out, visit any page and click the logout option in the navigation. This calls `POST /api/auth/logout` which clears the `ms_token` cookie.

4.3 JWT & Session Management

PROPERTY	VALUE
Cookie name	`ms_token`
Cookie flags	httpOnly, Secure (production), SameSite=Lax
Token expiry	7 days
JWT payload	`{ sub: userId, email, name, iat, exp }`
Signing algorithm	HS256 (HMAC-SHA256)

PROPERTY	VALUE
Library	`jose`

API route handlers extract the session using `lib/session.ts`:

```
import { getSession } from '@/lib/session';

export async function GET(request: NextRequest) {
  const session = await getSession(request);
  if (!session) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }
  // session.sub = userId
}
```

4.4 Route Protection

The `proxy.ts` middleware intercepts every HTTP request and enforces authentication:

ROUTE PATTERN	PROTECTION	BEHAVIOUR
`/login`, `/signup`	Public	Always accessible; logged-in users are redirected to `/india`
`/api/auth/*`	Public	Auth endpoints (login, signup, logout, me)
`/api/upstox/*`	Public	Upstox OAuth callback + status
`/api/alerts/check`	Public	Background alert trigger endpoint
`/api/cron/*`	Public (Bearer)	Cron prefetch — validated by `CRON_SECRET` header, not JWT cookie
`/api/public/*`	Public	Unauthenticated data endpoints (e.g., login page NSE preview)
`/_next/*`, `/favicon.ico`	Public	Static assets
All other routes	PROTECTED	Requires valid `ms_token` cookie; missing/expired → redirect to `/login`

5 Market Watchlists

Real-time price display for India, US, and Hong Kong markets

5.1 India (NSE) Watchlist — `/india`

The India watchlist connects to **Upstox** via an authorized WebSocket feed and displays live NSE prices in Indian Rupees (₹).

Available Instruments

SYMBOL	COMPANY	UPSTOX INSTRUMENT KEY
RELIANCE	Reliance Industries	`NSE_EQ|INE002A01018`
TCS	Tata Consultancy Services	`NSE_EQ|INE467B01029`
INFY	Infosys	`NSE_EQ|INE009A01021`
HDFCBANK	HDFC Bank	`NSE_EQ|INE040A01034`
ICICIBANK	ICICI Bank	`NSE_EQ|INE090A01021`
WIPRO	Wipro	`NSE_EQ|INE075A01022`
ITC	ITC Limited	`NSE_EQ|INE154A01025`
SBIN	State Bank of India	`NSE_EQ|INE062A01020`

Upstox WebSocket Data Flow

The India watchlist uses a persistent WebSocket connection running entirely in the browser. Vercel serverless functions are not involved in the live stream — only in the initial token exchange.

1. **Auth check** — On component mount, `useUpstoxStream` calls `GET /api/upstox/status` to verify that a valid `UPSTOX_ACCESS_TOKEN` is configured server-side.

2. **WebSocket URL** — When authenticated, `GET /api/upstox/ws-url` is called. The server uses the access token to request an authorized WSS URL from Upstox's API, then returns it to the client. This keeps the token off the browser.

3. **Binary stream** — The browser opens a direct `WebSocket` to Upstox's servers. Messages arrive as **protobuf-encoded binary frames** (not JSON). The `lib/upstox-tick-data/proto.ts` decoder (lazy-loaded) parses each frame.

4. **Subscription** — After connecting, the client sends a JSON subscription message (encoded as UTF-8 bytes) to subscribe to `DEFAULT_INSTRUMENT_KEYS` in `full` mode (LTP + bid/ask + volume).

5. **React state** — Each decoded frame updates the `ticks` React state map (`instrumentKey → UpstoxTick`), instantly re-rendering the table row for that symbol.

6. **Price history** — Each incoming LTP is appended to a rolling history buffer (max 300 points per symbol). This buffer powers the sparkline mini-charts in the Chart column.

7. **Alert check** — On every tick update, `POST /api/alerts/check` is called with the new price to evaluate and trigger any matching ACTIVE price alerts.

8. **Auto-reconnect** — If the WebSocket closes (network drop, Upstox server restart), the client automatically reconnects after 5 seconds. The subscription is re-sent on reconnect.

> ℹ️ **Works on Vercel Hobby Plan**
>
> Because the WebSocket runs in the browser (not on a server), Vercel's serverless environment is not a limitation. The only server calls are the initial status check and WSS URL fetch — both are short-lived HTTP requests, not persistent connections.

Login Page Live NSE Preview

The login page (`/login`) shows a live NSE data panel on the left side — visible before a user signs in. This is served by the public endpoint `GET /api/public/india-preview` (no authentication required).

- Reads the last 2 daily candles per India instrument from TimescaleDB to compute price, change, and change %.
- Returns the most recent `fetchedAt` timestamp — shown as "Updated 15 Apr 18:01 IST".
- The login page polls this endpoint every **5 minutes** (`setInterval`) so data refreshes automatically after each cron run without a redeploy.
- Cache-Control is set to `no-store` so Vercel's CDN does not cache stale data.

> ⚠️ **Market Hours**
>
> NSE trading hours are 9:15 AM – 3:30 PM IST (Mon–Fri). Intraday (1m–1h) candles will be empty outside these hours. The chart auto-falls back to the 1-day interval when the market is closed.

5.2 US Market Watchlist — `/us`

The US watchlist connects to **AllTick** WebSocket and displays live NYSE/NASDAQ prices in USD ($).

Available Instruments

AAPL · MSFT · NVDA · TSLA · AMZN · GOOGL · META · UNH

Each instrument is identified by the AllTick code format: `AAPL.US`, `MSFT.US`, etc.

Watchlist Columns

- **Symbol** — Ticker code and company name
- **Mid Price** — Average of bid and ask
- **Bid / Ask** — Best bid and ask from order book
- **Spread** — Ask minus bid
- **Chart icon** — Opens the candlestick chart modal
- **Buy / Sell buttons** — Opens the order form for that symbol

5.3 Hong Kong Watchlist — `/hk`

Displays HKEX prices in HKD (HK$). Uses the same AllTick WebSocket connection as the US market.

Available Instruments

700.HK (Tencent) · 9988.HK (Alibaba) · 1299.HK (AIA Group) · 0005.HK (HSBC) · 0939.HK (CCB) · 2318.HK (Ping An) · 0388.HK (HKEX) · 1211.HK (BYD)

> ℹ️ **HK Interval Limitations**
> Due to AllTick tier limitations for HKEX, only 1m, 5m, and 15m intervals are available for Hong Kong instruments. The chart automatically uses aggregation (e.g., 1m → 5m) when the provider does not support the requested interval directly.

5.4 Candlestick Charts

Click the chart icon on any instrument row to open a full-screen candlestick chart modal.

Chart Controls

CONTROL	OPTIONS	NOTES
Interval	1m · 5m · 15m · 30m · 1h · 1d	HK limited to 1m/5m/15m
Period	1W · 1M · 3M · 6M · 1Y	Applies a historical lookback filter

Cache Behaviour

INTERVAL	CACHE TTL (STALE AFTER)
1m	1 minute
5m	5 minutes
15m	15 minutes
30m	30 minutes

INTERVAL	CACHE TTL (STALE AFTER)
1h	1 hour
1d	12 hours

When TimescaleDB is configured, the chart first checks if cached candles are fresh. If fresh, they are served from DB. If stale or absent, fresh candles are fetched from the data provider and written back to TimescaleDB.

6 Paper Trading Engine

Placing orders, execution logic, and position tracking

6.1 Placing Orders

Navigate to **/orders** to place trades. Orders can also be initiated by clicking **Buy** or **Sell** on any watchlist row, which pre-fills the order form.

Order Form Fields

Market	india / us / hk
Symbol	Instrument code (e.g., RELIANCE, AAPL)
Side	BUY or SELL
Order Type	MARKET (execute immediately) or LIMIT (wait for price)
Quantity	Number of units to buy or sell
Price	Limit price (for LIMIT orders) or current market price (for MARKET)
Notes	Optional memo / strategy note

6.2 Order Types & Statuses

Order Types

- **MARKET** — Executes instantly at the current price. Balance is deducted / credited immediately.
- **LIMIT** — Saved as PENDING. Not executed until manually filled or a matching price trigger occurs.

Order Statuses

- PENDING — Awaiting execution (LIMIT)
- FILLED — Fully executed
- PARTIAL — Partially filled
- CANCELLED — Cancelled by user
- REJECTED — Insufficient funds or position

6.3 Order Execution Logic

The paper trading engine lives in `lib/trading.ts`. Here is how each trade is processed:

BUY Order

1. Calculate total cost: `quantity × price`.
2. Check user balance is sufficient. If not → REJECTED.
3. Deduct cost from user balance.
4. Create or update a `Position` record using **weighted-average cost**:
   `newAvgCost = (oldQty × oldAvgCost + newQty × price) / (oldQty + newQty)`
5. Mark order as FILLED with `avgFillPrice` and `filledAt` timestamp.

SELL Order

1. Check that a `Position` exists for this symbol with sufficient quantity. If not → REJECTED.
2. Credit proceeds (`quantity × price`) to user balance.
3. Calculate realized P&L: `(price - avgCost) × quantity`.
4. Create a `PnlEntry` record for the realized gain/loss.
5. Reduce position quantity. If position reaches zero → delete position record.
6. Mark order as FILLED.

> ℹ️ **No Interactive Transactions**
>
> Prisma Postgres does not support interactive transactions via the pg adapter. All write operations are performed as sequential queries rather than wrapped in a single database transaction.

7 Portfolio Management

View positions, track P&L, and monitor total portfolio value

7.1 Positions & Holdings

Navigate to **/portfolio** to see your current holdings. The portfolio dashboard displays:

- **Cash Balance** — Remaining paper money (starts at ₹1,00,000)
- **Open Positions** — Each stock held with current market value
- **Total Portfolio Value** — Cash + sum of all position market values
- **Market Allocation** — Breakdown by India / US / HK

Positions Table Columns

COLUMN	DESCRIPTION
Symbol	Instrument code and market label
Quantity	Total units held
Avg Cost	Weighted-average purchase price
Current Price	Latest live price from feed
Market Value	quantity × current price
Unrealized P&L	(current price − avg cost) × quantity
P&L %	(unrealized P&L / cost basis) × 100

7.2 P&L Calculation

Unrealized P&L

Calculated live using current market price:

```
pnl = (price - avgCost) × qty
pnl% = (pnl / costBasis) × 100
```

Updates in real-time as prices change in the WebSocket feed. Shown in green (profit) or red (loss).

Realized P&L

Recorded when a SELL order executes:

```
realizedPnl =
  (salePrice - costBasis)
  × quantity
```

Each sale creates a permanent `PnlEntry` record. The Realized P&L table shows full sell history.

Portfolio API Response

`GET /api/portfolio` returns:

```
{
  "balance": 85000.00,          // remaining cash
  "positions": [
    {
      "symbol": "RELIANCE",
      "market": "india",
      "quantity": 10,
      "avgCost": 2450.00,
      "currentPrice": 2510.50,  // from live feed
      "marketValue": 25105.00,
      "unrealizedPnl": 605.00,
      "pnlPct": 2.47
    }
  ],
  "pnlEntries": [...],          // realized P&L history
  "totals": {
    "costBasis": 24500.00,
    "marketValue": 25105.00,
    "unrealizedPnl": 605.00,
    "totalValue": 110105.00     // cash + market value
  }
}
```

8 Price Alerts

Set price triggers for automatic notifications or trade execution

Navigate to **/alerts** to manage price alerts. Alerts monitor price levels and fire notifications or auto-execute trades when conditions are met.

Creating an Alert

FIELD	OPTIONS	DESCRIPTION
Symbol	Any tracked instrument	The stock to monitor
Market	india / us / hk	Which exchange
Condition	ABOVE · BELOW · EQUAL	Price direction trigger
Target Price	Numeric	Price level that triggers the alert
Action	NOTIFY · BUY · SELL	What happens when alert fires
Quantity	Numeric (required for BUY/SELL)	Number of units to trade on trigger

Alert Conditions

Trigger Logic

- **ABOVE** — fires when `currentPrice > targetPrice`
- **BELOW** — fires when `currentPrice < targetPrice`
- **EQUAL** — fires when price is within ±0.5% of target

Alert Actions

- **NOTIFY** — browser notification (no trade)
- **BUY** — auto-executes a MARKET buy order
- **SELL** — auto-executes a MARKET sell order

Alert Status Lifecycle

STATUS	MEANING
ACTIVE	Monitoring — will fire when price condition is met
TRIGGERED	Condition was met; action was taken. Stored for history.
CANCELLED	Manually cancelled by user before triggering

Alert Trigger Endpoint

The endpoint `POST /api/alerts/check` accepts a price payload and evaluates all ACTIVE alerts for matching symbols. It is designed to be called by an external price-checking service or scheduled job.

```
// POST /api/alerts/check
{
  "symbol": "RELIANCE",
  "market": "india",
  "price": 2500.00
}
```

> ⚠️ **Auto-Trade Alerts**
>
> When an alert with action BUY or SELL triggers, it executes a real paper trade on your account. Ensure your balance and positions are adequate before setting BUY/SELL alerts.

9 User Profile

View account stats, update name, and change password

Navigate to **/profile** (or click the avatar in the top-right navigation) to manage your account.

Profile Page Sections

Account Overview (Stat Cards)

STAT CARD	DESCRIPTION
Cash Balance	Current paper money available (e.g., ₹85,000.00). Displayed on a single line with monospace font.
Total Orders	Cumulative count of all orders placed (all statuses)
Open Positions	Count of instruments currently held in portfolio
Active Alerts	Count of ACTIVE price alerts currently monitoring

Edit Name

Click the pencil icon next to your displayed name to enter edit mode. Type a new name and click **Save**. The JWT cookie is refreshed with the new name so the navigation avatar updates immediately.

Account Details (Read-only)

- **Email** — the address used to sign up (cannot be changed)
- **User ID** — internal CUID identifier
- **Account Type** — Paper Trading Account
- **Member Since** — account creation date

Change Password

The password change form requires three fields:

1. **Current Password** — verified against stored bcrypt hash
2. **New Password** — the replacement password
3. **Confirm New Password** — must match new password exactly

On success, the new password hash is stored and a success message is shown. The session cookie is not invalidated (the user remains logged in).

Profile API

ENDPOINT	METHOD	DESCRIPTION
`/api/profile`	GET	Returns user info + stats (order count, positions, active alerts)
`/api/profile`	PATCH	Update name (body: `{name}`) or password (body: `{currentPassword, newPassword}`)

10 API Reference

All HTTP endpoints with request/response details

Authentication Endpoints

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	`/api/auth/signup`	Public	Register new user. Body: `{name, email, password}`
POST	`/api/auth/login`	Public	Authenticate. Body: `{email, password}`. Sets `ms_token` cookie.
POST	`/api/auth/logout`	Public	Clears `ms_token` cookie.
GET	`/api/auth/me`	Required	Returns current user info from JWT.

Order Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/orders`	List orders. Query: `?status=FILLED&symbol=RELIANCE&market=india`
POST	`/api/orders`	Place order. Body: `{symbol, market, label, side, orderType, quantity, price, notes?}`
GET	`/api/orders/[id]`	Get single order by ID.
PATCH	`/api/orders/[id]`	Update order (e.g., fill a LIMIT order or cancel).

POST /api/orders — Request Body

```
{
  "symbol":         "RELIANCE",
  "market":         "india",
  "label":          "RELIANCE",
  "currencySymbol": "₹",
  "side":           "BUY",          // "BUY" | "SELL"
  "orderType":      "MARKET",       // "MARKET" | "LIMIT"
  "quantity":       10,
  "price":          2450.00,
  "notes":          "Optional note"
}
```

Portfolio Endpoint

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/portfolio`	Returns balance, all positions with live P&L, realized P&L history, and totals.

Alert Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/alerts`	List alerts. Query: `?status=ACTIVE`
POST	`/api/alerts`	Create alert. Body: `{symbol, market, label, condition, targetPrice, action, quantity?}`
PATCH	`/api/alerts/[id]`	Update alert (e.g., cancel).
POST	`/api/alerts/check`	Check & trigger alerts. Body: `{symbol, market, price}`

Market Data (Candlestick) Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/india/kline`	India candles. Query: `?instrumentKey=NSE_EQ|INE002A01018&interval=1d&period=1m`
GET	`/api/us/kline`	US/HK candles. Query: `?code=AAPL.US&interval=5m&period=1w&market=us`

Kline API Parameters

PARAMETER	REQUIRED	VALUES
`interval`	Yes	1m · 5m · 15m · 30m · 1h · 1d
`period`	No	1w · 1m · 3m · 6m · 1y (historical range)
`market`	US/HK only	us · hk
`limit`	No	Max candles (default: 300, max: 1500)

Kline API Response

```
{
  "candles": [
    {
      "time":   1713000000,   // Unix timestamp (seconds)
```

```
      "open":   2445.00,
      "high":   2460.50,
      "low":    2440.00,
      "close":  2455.75,
      "volume": 125000
    }
  ],
  "source": "db"   // "db" | "live" | "db-fallback"
}
```

Profile Endpoint

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/profile`	Returns `{id, name, email, balance, createdAt, stats: {orders, positions, activeAlerts}}`
PATCH	`/api/profile`	Update name: `{name}`. Change password: `{currentPassword, newPassword}`.

11 Data Providers & Market Data

How live prices and historical candles are sourced

Provider Overview

MARKET	PROVIDER	PROTOCOL	DATA TYPE
India (NSE)	Upstox	WebSocket (protobuf) + REST v2	Live ticks + historical OHLCV
US (NYSE/NASDAQ)	AllTick	WebSocket (JSON) + REST	Live order book + historical OHLCV
Hong Kong (HKEX)	AllTick	WebSocket (JSON) + REST	Live order book + historical OHLCV

Upstox (India)

Live Price Feed

- OAuth 2.0 authorization generates a bearer token stored as `UPSTOX_ACCESS_TOKEN`.
- The token is used to request an authorized WebSocket URL from `/api/upstox/ws-url`.
- Price updates arrive as **protobuf-encoded** binary messages, decoded by `lib/upstox-tick-data/`.
- The decoded tick includes: LTP (last traded price), bid, ask, volume, OI.

Historical OHLCV

Upstox provides two REST endpoints used by `/api/india/kline`:

ENDPOINT TYPE	URL PATTERN	USED FOR
Historical	`/v2/historical-candle/{instrument}/{unit}/{toDate}/{fromDate}`	Multi-day / multi-week historical data (1d, 1w, 1mo intervals)
Intraday	`/v2/historical-candle/intraday/{instrument}/{unit}`	Today's intraday candles only (1m–1h when no period specified)

> ℹ️ **Upstox Data Direction**
>
> Upstox returns candles in newest-first order. The kline route reverses the array to ascending chronological order before returning or caching.

AllTick (US / HK)

Live Price Feed

- AllTick provides a single WebSocket endpoint for US and HK markets.
- Messages are JSON with order book depth (bid/ask levels).
- A heartbeat mechanism keeps the connection alive. Auto-reconnect on disconnect.
- Up to 300 price history points are maintained per symbol for sparklines.

Historical OHLCV (REST)

```
GET https://quote.alltick.co/quote-stock-b-api/kline
  ?token=YOUR_TOKEN
  &query={"trace":"id","data":{"code":"AAPL.US","kline_type":5,"query_kline_num":300}}
```

`kline_type` is the interval in minutes: 1=1m, 5=5m, 15=15m, 30=30m, 60=1h, 1440=1d.

Fallback Aggregation

If AllTick does not provide the requested interval (common with HK instruments on lower API tiers), the system automatically:

1. Fetches a finer-grained interval (e.g., 1m data for a 5m request).
2. Validates the cadence of returned data.
3. Aggregates candles into the target interval using OHLCV bucket logic.

TimescaleDB Candle Storage

The `lib/timescale.ts` module manages all TimescaleDB interactions:

FUNCTION	SQL OPERATION	PURPOSE
`getNewestFetchedAt()`	`SELECT MAX(fetched_at)`	Check when candles were last cached (freshness check)
`readCandles()`	`SELECT ... WHERE time >= cutoff ORDER BY time ASC LIMIT n`	Read cached candles from hypertable
`writeCandles()`	`INSERT ... ON CONFLICT DO UPDATE SET open=EXCLUDED.open, ...`	Bulk upsert via unnest() arrays; updates existing candle on conflict so today's daily candle refreshes on every cron run

Candle Table Schema (TimescaleDB)

```
CREATE TABLE "Candle" (
  market   TEXT    NOT NULL,    -- 'india' | 'us' | 'hk'
```

```sql
  symbol   TEXT    NOT NULL,    -- instrument code
  interval TEXT    NOT NULL,    -- '1m' | '5m' | ... | '1d'
  time     INTEGER NOT NULL,    -- Unix timestamp (seconds)
  open     NUMERIC NOT NULL,
  high     NUMERIC NOT NULL,
  low      NUMERIC NOT NULL,
  close    NUMERIC NOT NULL,
  volume   NUMERIC NOT NULL,
  "fetchedAt" TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (market, symbol, interval, time)
);

SELECT create_hypertable('"Candle"', 'time',
  chunk_time_interval => 2592000);   -- 30-day chunks

SELECT add_compression_policy('"Candle"',
  compress_after => 7776000);        -- compress after 90 days
```

12 Deployment (Vercel)

Deploying MarketSync to production on Vercel

Deployment Prerequisites

- Vercel CLI installed: `npm install -g vercel`
- All databases created and initialized (Prisma Postgres + TimescaleDB)
- API tokens for Upstox and AllTick obtained

Step-by-Step Deployment

1. **Link to Vercel project**
   `vercel link` (follow prompts to connect to existing project)

2. **Set environment variables via CLI**

```
printf "postgres://timescale..." | vercel env add DATABASE_URL production
printf "your-jwt-secret" | vercel env add JWT_SECRET production
printf "your-cron-secret" | vercel env add CRON_SECRET production
printf "upstox-token" | vercel env add UPSTOX_ACCESS_TOKEN production
printf "0" | vercel env add NODE_TLS_REJECT_UNAUTHORIZED production
printf "alltick-token" | vercel env add NEXT_PUBLIC_ALLTICK_TOKEN production
```

   Note: use `printf` (not `echo`) to avoid trailing newlines in the stored value.

3. **Deploy to production**
   `vercel --prod`

4. **Set up GitHub Actions cron secrets**
   In the GitHub repository → Settings → Secrets and variables → Actions, add:
   `CRON_SECRET` — same value as the Vercel env var
   `VERCEL_URL` — your Vercel deployment URL (e.g. `marketsync-rho.vercel.app`)

Build Script

The `package.json` build command is:

```
"build": "prisma generate && next build"
```

This ensures the Prisma client is generated from the schema before the Next.js build runs. The generated client is not committed to git — it is regenerated at build time on every deploy.

Vercel Environment Variables Summary

VARIABLE	ENVIRONMENT	SOURCE
`DATABASE_URL`	Production, Preview	TigerData Cloud connection string
`JWT_SECRET`	Production, Preview	Self-generated (min 32 chars)
`CRON_SECRET`	Production	Self-generated; also stored as GitHub Actions secret
`UPSTOX_ACCESS_TOKEN`	Production	Upstox OAuth flow (refresh daily)
`NODE_TLS_REJECT_UNAUTHORIZED`	Production	Set to `0` — bypasses Upstox TLS certificate
`NEXT_PUBLIC_ALLTICK_TOKEN`	Production	AllTick API dashboard

GitHub Actions Cron Job

Because Vercel Hobby plan does not support cron schedules more frequent than daily, candle prefetching is handled by a GitHub Actions scheduled workflow.

File: `.github/workflows/prefetch-candles.yml`

PROPERTY	VALUE
Schedule	Every 5 minutes (`*/5 * * * *`)
Trigger	Also supports manual `workflow_dispatch`
Action	`curl -s -L` to `GET /api/cron/prefetch` with `Authorization: Bearer $CRON_SECRET`
Timeout	4 minutes per run
Markets	India (1d via Upstox), US (1d via AllTick), HK (1d via AllTick)
Rate limit	5 second delay between AllTick API calls (free plan)

> ℹ️ **GitHub Free Plan Cron Accuracy**
>
> GitHub Actions free plan does not guarantee sub-hourly cron execution. Actual run frequency may be every 1–3 hours depending on GitHub's queue load. Use `workflow_dispatch` (manual trigger via GitHub API or UI) to force an immediate prefetch.

> 🔒 **Security Warning**
>
> `NEXT_PUBLIC_` prefixed variables are exposed to the browser. The AllTick token uses this prefix because it is needed in client-side WebSocket connections. Never use `NEXT_PUBLIC_` for sensitive secrets like JWT_SECRET or database credentials.

Updating Upstox Token

The Upstox access token expires daily. To refresh it:

1. Complete the Upstox OAuth flow (click **Connect Upstox** on the India page).
2. The new token is received and stored server-side.
3. Update the `UPSTOX_ACCESS_TOKEN` in Vercel: `vercel env rm UPSTOX_ACCESS_TOKEN production` then re-add.

13 Troubleshooting

Common issues and how to resolve them

SYMPTOM	LIKELY CAUSE	SOLUTION
Blank candlestick chart (intraday)	NSE market is closed; intraday returns no candles	Chart auto-falls back to 1d interval. This is expected behavior outside 9:15–15:30 IST.
HTTP 502 on chart load	Upstox / AllTick API returned an error	Chart falls back to 1d interval automatically. Check UPSTOX_ACCESS_TOKEN validity. Upstox tokens expire daily.
India watchlist shows no prices	Upstox token missing or expired	Click **Connect Upstox** button and complete OAuth. Update UPSTOX_ACCESS_TOKEN env var.
US/HK watchlist shows no prices	AllTick token invalid or quota exceeded	Verify NEXT_PUBLIC_ALLTICK_TOKEN in .env. Check AllTick dashboard for quota usage.
Prisma migration error (P3018)	A previous migration failed partially	Run `npx prisma migrate resolve --rolled-back <migration_id>` then retry `npx prisma migrate deploy`.
Module not found: ./generated/prisma/client	Prisma generate not run before build	Ensure build script is `"prisma generate && next build"` in package.json.
Candles not caching to TimescaleDB	DATABASE_URL is empty or invalid	Verify DATABASE_URL points to TimescaleDB instance. Run `scripts/init-timescale.sql` to create the Candle hypertable if missing.
Cron job returns HTTP 401	CRON_SECRET mismatch or newline in stored value	Use `printf` (not `echo`) when setting Vercel env vars. Ensure GitHub Actions secret matches exactly.
Cron job returns HTTP 307 (redirect)	curl not following redirects	Ensure `curl -L` flag is used in the workflow to follow HTTP → HTTPS redirects.
AllTick returns HTTP 429 during cron	Rate limit exceeded on free plan	Cron uses 5-second delay between AllTick calls and daily-only intervals. If still hitting limits, increase delay in `/api/cron/prefetch/route.ts`.
Upstox SSL error during cron	Upstox API has self-signed TLS certificate	Ensure `NODE_TLS_REJECT_UNAUTHORIZED=0` is set in Vercel environment variables.

SYMPTOM	LIKELY CAUSE	SOLUTION
Login page shows stale NSE data	Previously caused by Vercel CDN caching the preview API	API now returns `Cache-Control: no-store`. Login page auto-polls every 5 min. Data updates within one cron cycle after market close.
Order rejected with "Insufficient balance"	Cash balance too low for the trade	Reduce quantity or check portfolio balance. Starting balance is ₹1,00,000 / $100,000.
SELL order rejected	No open position for that symbol	Ensure you have previously bought the instrument. Check /portfolio for open positions.
Login redirect loop	JWT_SECRET mismatch between env and stored cookies	Clear browser cookies. Ensure JWT_SECRET is consistent across deploys.
ConnectionBadge shows Disconnected	WebSocket connection dropped	The client auto-reconnects. If persistent, refresh the page. Check network / firewall for WSS blocking.
PowerShell: taskkill /PID error	Git Bash mangles /PID flag	Use: `powershell -Command "Stop-Process -Id <PID> -Force"`

Useful Commands

```
# Check Upstox token status
curl http://localhost:3000/api/upstox/status

# Manually trigger candle prefetch (local)
curl -H "Authorization: Bearer YOUR_CRON_SECRET" http://localhost:3000/api/cron/prefetch

# Check login page NSE preview data
curl http://localhost:3000/api/public/india-preview

# View Prisma DB in browser
npx prisma studio

# Regenerate Prisma client
npx prisma generate

# Apply pending migrations
npx prisma migrate deploy

# View Vercel deployment logs
vercel logs

# List Vercel env vars
vercel env ls

# Redeploy to production
```

```
vercel --prod
```

Production Status (April 2026)

✅ **Single TimescaleDB — Fully Configured**

All app tables and the Candle hypertable are stored in a single TimescaleDB instance on TigerData Cloud. `DATABASE_URL` points to this instance in Vercel. Both Prisma ORM (app tables) and raw `pg` Pool (candles) connect via the same connection string.

ℹ️ **Upstox Token Refresh**

The Upstox access token expires daily. When it expires, the India watchlist will show "Connect" state. Re-authenticate via the **Connect Upstox** button, then update `UPSTOX_ACCESS_TOKEN` in Vercel and redeploy (or update via CLI: `printf "new-token" | vercel env add UPSTOX_ACCESS_TOKEN production`).